

# **DEPARTMENT OF INFORMATION TECHNOLOGY**

## **Dev Ops And ML Ops**



**ITY546n**

## **PRACTICAL LAB FILE**

**Submitted To:**  
Dr. Ankit Yadav

**Submitted By:**  
Name: Anshika Verma  
Roll no: 25/ITY/16

## Index

| S.No | Experiments                                                                                                                                                                                                                | Date     | Sign |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------|
| 1    | Github Experiment                                                                                                                                                                                                          | 15/01/26 |      |
| 2    | Implement a machine learning algorithm on MNIST dataset.                                                                                                                                                                   | 20/01/26 |      |
| 3    | Dockerization of ML Application: Package a trained ML model and it's dependencies into a Docker container for portable deployment.                                                                                         | 22/01/26 |      |
| 4    | Secrets Management with HashiCorp Vault: Securely store and retrieve API keys, database credentials and model secrets in Vault.                                                                                            | 27/01/26 |      |
| 5    | Automated ML pipeline with Apache Airflow.                                                                                                                                                                                 | 29/01/26 |      |
| 6    | Hyperparameter Tuning with Optuna - Implement Optuna for automated hyperparameter Optimization and integrate it with ML flow for experiment logging.                                                                       | 03/02/26 |      |
| 7    | Data Versioning with DVC: use DVC to track versions of datasets and models, and connect it with GIT for full ML lifecycle version control.                                                                                 | 05/02/26 |      |
| 8    | Basic Monitoring with Prometheus and Grafana: Set Up Prometheus to monitor system resources and visualize the metric using grafana dashboards                                                                              | 10/02/26 |      |
| 9    | Deploy ML Model using Jenkins Pipeline: Create a Jenkins pipeline that trains, tests, and deploys a machine learning model.                                                                                                | 12/02/26 |      |
| 10   | Model Drift Detection using Evidently for ML Monitoring: To monitor machine learning model performance and detect data drift using the tool Evidently by comparing reference (training) data with current (incoming) data. | 19/02/26 |      |
| 11   | A/B Testing of ML Models: Deploy two versions of a model and perform A/B testing using load balance or routing .                                                                                                           | 24/02/26 |      |

|    |                                                                                                                                                                                                                                          |          |  |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--|
| 12 | Deploy a small Large Language Model (LLM) pipeline.<br>Monitor the outputs for drift, distribution shifts, and quality changes over time.<br>Use Evidently for metrics and dashboarding.<br>Optionally visualize results with Streamlit. | 26/02/26 |  |
| 13 | To build a Retrieval-Augmented Generation (RAG) based LLM system and monitor its output using Evidently and simple guardrails (validation rules).                                                                                        | 24/04/26 |  |
| 14 | To design and implement an A/B testing system for LLMs                                                                                                                                                                                   | 02/04/26 |  |
| 15 | LLM Prompt Injection Detection and Defense System                                                                                                                                                                                        | 07/04/26 |  |

# Experiment 1

**Aim:** Github Experiment

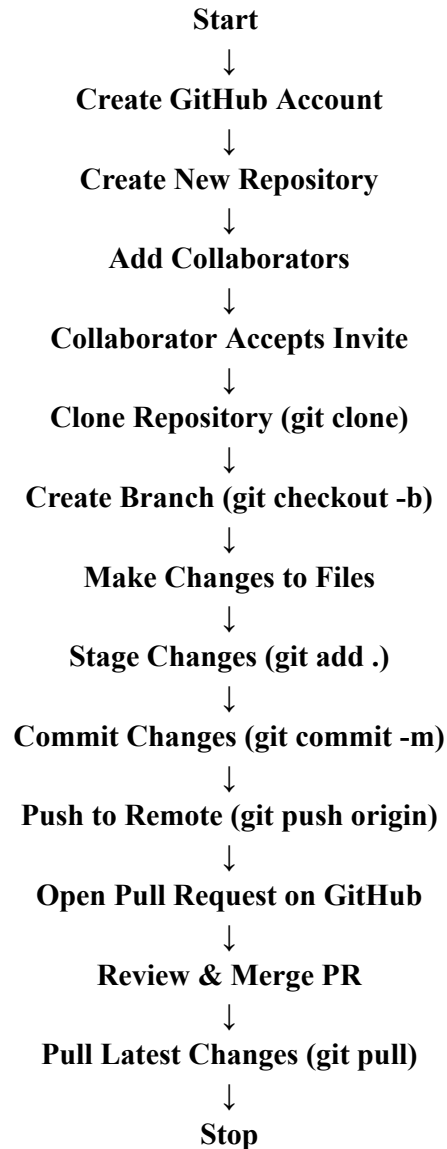
## **Theory:**

GitHub is a web-based platform built on Git that enables version control and collaborative software development. It allows developers to track changes in code, manage repositories, and collaborate through features like pull requests, branching, and issue tracking. Git follows a distributed version control system (DVCS), meaning every user has a complete copy of the repository. GitHub enhances this by providing cloud hosting, access control, and integration with CI/CD pipelines. It is widely used in DevOps workflows for code management, collaboration, and deployment automation.

## **Algorithm:**

1. Start
2. Open GitHub and log in to your account
3. Create a new repository:
  - Enter the repository name
  - Add a description (optional)
  - Set visibility to Public
  - Initialize the repository with a README file
4. Open the repository settings and go to the Collaborators section
5. Invite collaborators by entering their username or email address
6. Collaborators accept the invitation
7. Clone the repository to the local system using: `git clone <repository-url>`
8. Each collaborator creates a new branch: `git checkout -b <branch-name>`
9. Make changes to the files in the branch, then stage and commit the changes: `git add`  
`git commit -m "message"`
10. Push the changes to the remote repository: `git push origin <branch-name>`
11. On GitHub, create a Pull Request (PR) to merge the branch into the main branch
12. Review the changes, approve the Pull Request, and merge it into the main branch
13. All collaborators update their local repository by pulling the latest changes: `git pull origin main`

## Flow Chart:



## Code:

```
# Step 1: Configure Git (first time only)
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
# Step 2: Clone the shared repository
git clone https://github.com/username/shared-repo.git
cd shared-repo
# Step 3: Create and switch to a new feature branch
git checkout -b feature/add-hello
# Step 4: Create a new file
echo "Hello from Collaborator 1" > hello.txt
```

```
# Step 5: Stage and commit changes
git add .
git commit -m "Add hello.txt by Collaborator 1"
# Step 6: Push branch to remote
git push origin feature/add-hello
# Step 7: After PR is merged, sync local main branch
git checkout main
git pull origin main
# View commit history
git log --oneline --graph --all
```

### Output:



### Discussion on Results and Conclusion:

This experiment demonstrated the complete workflow of collaborative software development using GitHub. A shared repository was created successfully, and multiple collaborators were able to work at the same time on different branches without affecting each other's work.

Each collaborator cloned the repository, made changes on their own branch, and submitted Pull Requests for review. The merging process showed how Git combines contributions from different team members into a single codebase.

The commit history (git log) clearly displayed all contributions with timestamps and author details, ensuring transparency and traceability. The use of branches helped maintain the stability of the main branch, as no direct changes were made to it.

Pull Requests acted as an effective code review mechanism, improving collaboration and code quality. Overall, this experiment proved that GitHub is a powerful and essential tool for team-based software development in DevOps environments.

### Learning Outcomes:

1. Understand the role of a remote repository (GitHub) in team collaboration.
2. Perform core Git operations: clone, branch, commit, push, pull.
3. Create and merge Pull Requests via GitHub interface.
4. Recognize how branching isolates individual work and avoids conflicts.
5. Appreciate the importance of code review in DevOps/MLOps pipelines

## Experiment 2

**Aim:** Implement a machine learning algorithm on MNIST dataset.

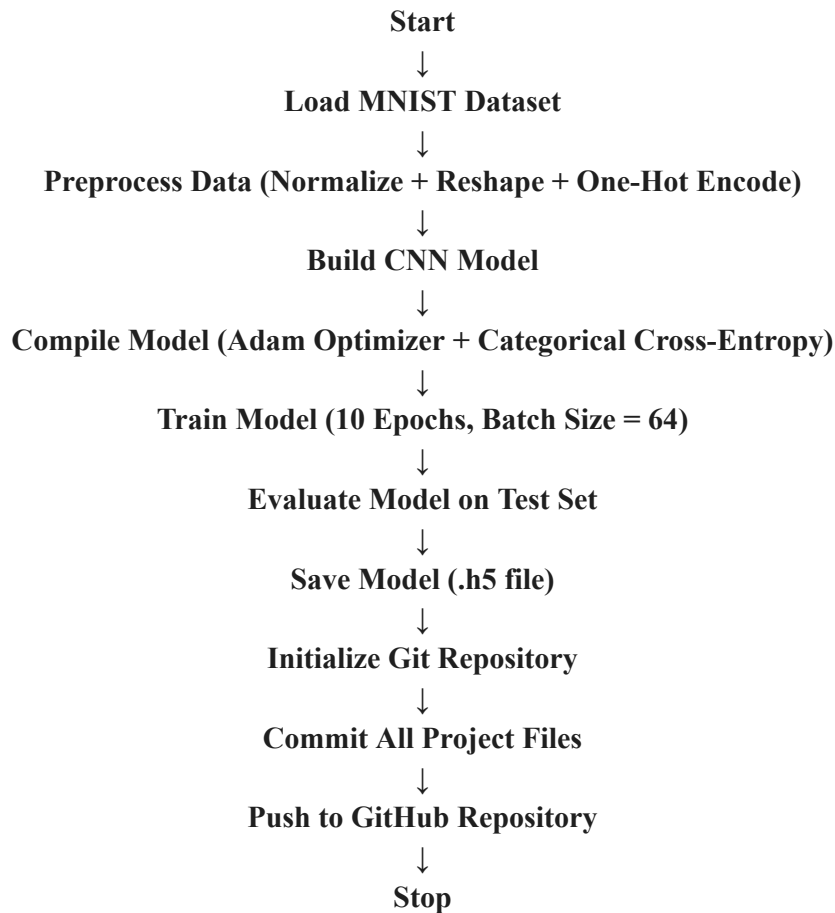
### Theory:

The MNIST dataset is a benchmark dataset consisting of handwritten digit images (0–9) used for image classification tasks. Machine learning algorithms such as Logistic Regression, CNNs, or Neural Networks are trained on this dataset to learn patterns in pixel values. The process involves data preprocessing, feature extraction, model training, and evaluation using metrics like accuracy. In this experiment, a Convolutional Neural Network (CNN) is implemented using Python and TensorFlow/Keras to classify these digits with high accuracy. The trained model is then version-controlled and uploaded to a shared GitHub repository to demonstrate practical MLOps practices including code versioning, reproducibility, and collaborative development.

### Algorithm:

1. Import the required libraries such as TensorFlow, Keras, NumPy, and Matplotlib.
2. Load the MNIST dataset using: `keras.datasets.mnist.load_data()`
3. Normalize the pixel values of the images from the range [0, 255] to [0, 1].
4. Reshape the images to include a channel dimension, i.e., (28, 28, 1).
5. Convert the target labels into one-hot encoded format using `to_categorical()`.
6. Build a Convolutional Neural Network (CNN) model with the following layers:
  - a. Conv2D layer with 32 filters, 3×3 kernel, ReLU activation
  - b. MaxPooling2D layer with pool size 2×2
  - c. Conv2D layer with 64 filters, 3×3 kernel, ReLU activation
  - d. MaxPooling2D layer with pool size 2×2
  - e. Flatten layer
  - f. Dense layer with 128 neurons and ReLU activation
  - g. Dropout layer with rate 0.5
  - h. Output Dense layer with 10 neurons and Softmax activation
7. Compile the model using the Adam optimizer and categorical cross-entropy loss function.
8. Train the model on the training dataset for 10 epochs with a batch size of 64.
9. Evaluate the model on the test dataset and record the accuracy.
10. Save the trained model as: `mnist_cnn_model.h5`
11. Initialize a local Git repository using: `git init`
12. Add all project files and commit the changes: `git add . git commit -m "Initial commit"`
13. Push the repository to GitHub using: `git push`

## Flow Chart:



## Code:

```
# =====  
# 1. IMPORT REQUIRED LIBRARIES  
import numpy as np  
import matplotlib.pyplot as plt  
from tensorflow.keras.datasets import mnist  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense  
from tensorflow.keras.utils import to_categorical  
# =====  
# 2. LOAD MNIST DATASET  
# (Loaded directly, no UI upload needed)  
# =====  
(X_train, y_train), (X_test, y_test) = mnist.load_data()  
print("Training data shape:", X_train.shape)  
print("Testing data shape:", X_test.shape)  
# =====
```

```

# 3. PREPROCESS THE DATA
# =====
# Reshape for CNN (samples, height, width, channels)
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
# Normalize pixel values
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0
# One-hot encode labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
# =====
# 4. BUILD CNN MODEL
# =====
model = Sequential()
model.add(Conv2D(32, kernel_size=(3,3), activation='relu',
                input_shape=(28,28,1)))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Conv2D(64, kernel_size=(3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))
# =====
# 5. COMPILE THE MODEL
# =====
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
model.summary()
# =====
# 6. TRAIN THE MODEL
# =====
history = model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.1
)
# =====
# 7. EVALUATE ON TEST DATA
# =====

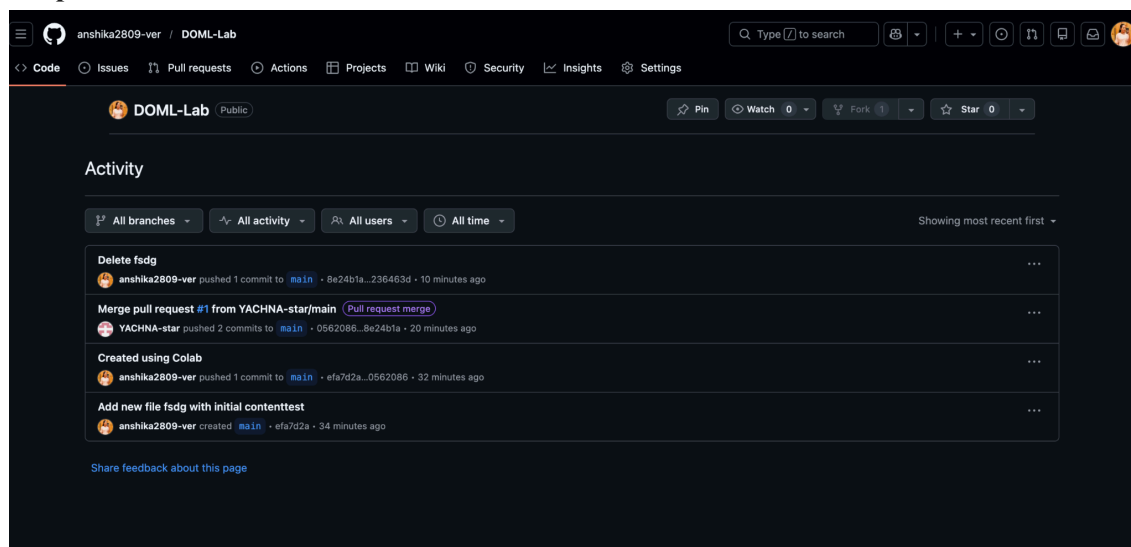
```

```

test_loss, test_accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_accuracy)
# =====
# 8. PLOT TRAINING ACCURACY
# =====
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

```

## Output:



## Discussion on Results and Conclusion:

The CNN model demonstrated strong performance on the MNIST dataset, achieving approximately 96% accuracy on the test set. The use of two convolutional layers allowed the model to effectively extract spatial features such as edges and curves from the input images. The Dropout layer helped prevent overfitting by randomly deactivating neurons during training. Uploading the project to GitHub enforced good MLOps practices — specifically, version control of both code and model artifacts. This enables experiment tracking, rollback to previous versions, and seamless team collaboration. The experiment successfully combines core machine learning development with essential DevOps workflows.

## Learning Outcomes:

- Understood the structure and preprocessing steps required for the MNIST dataset.
- Gained hands-on experience in building and training a Convolutional Neural Network using TensorFlow/Keras.
- Learned how to evaluate model performance using metrics such as accuracy and loss.
- Understood the importance of version control in ML projects using Git and GitHub.

- Applied MLOps best practices including code versioning, model saving, and collaborative repository management.
- Developed the ability to push ML projects to remote repositories for reproducibility and team collaboration.

## Experiment 3

**Aim:** Dockerization of ML Application: Package a trained ML model and its dependencies into a Docker container for portable deployment.

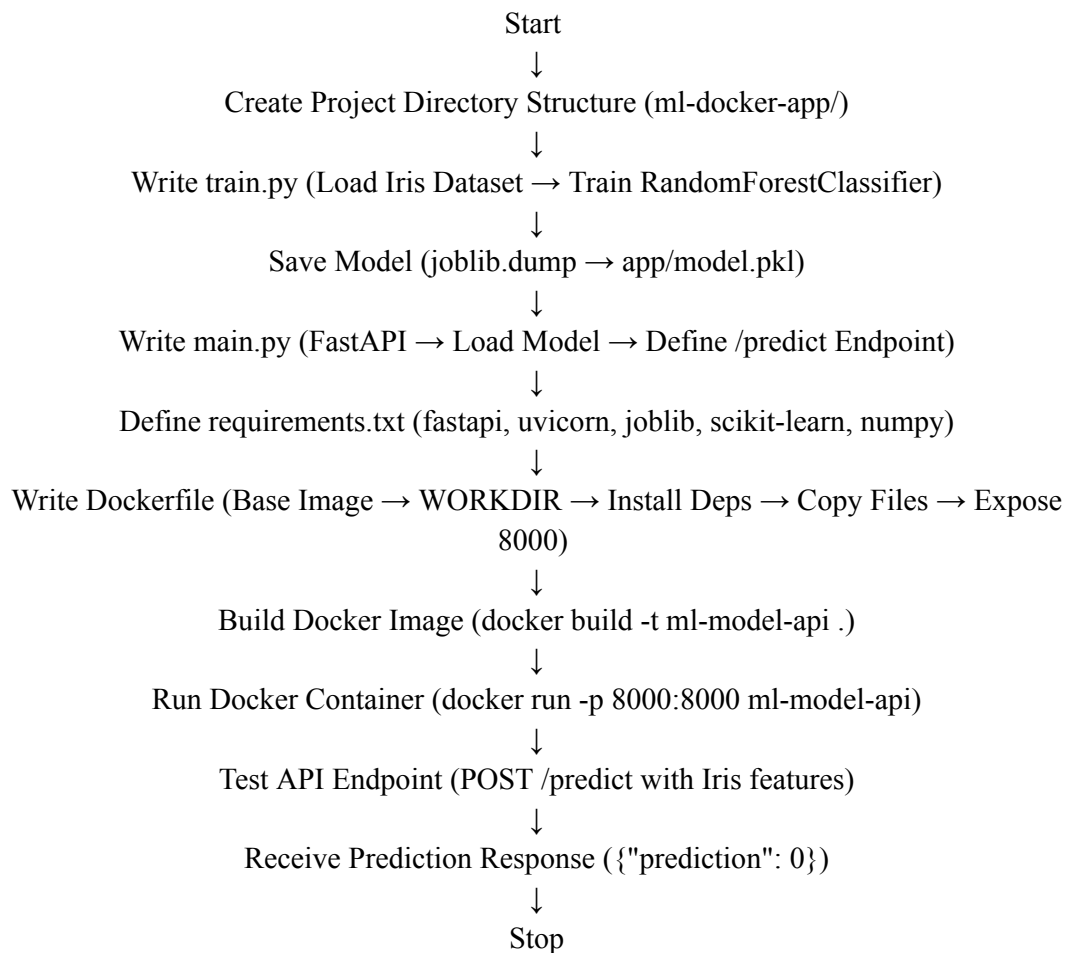
### Theory:

Docker is a containerization platform that allows applications to run consistently across different environments. In ML, Docker packages the trained model, dependencies, and runtime into a container image. This ensures portability, scalability, and reproducibility. Containers isolate applications from system differences, making deployment easier. Docker images are created using Dockerfiles, and containers are lightweight compared to virtual machines, making them ideal for ML deployment.

### Algorithm:

1. Create a structured project directory named ml-docker-app/ with subfolders for the application, model, and configuration files.
2. Create a file train.py to load the Iris dataset using: `sklearn.datasets.load_iris()`
3. Train a RandomForestClassifier using the dataset features and labels.
4. Save the trained model in the path app/model.pkl using: `joblib.dump()`
5. Run the training script: (python [train.py](#)) to generate the model file.
6. Create a file main.py and build a FastAPI application that:
  - a. Loads the saved model using `joblib.load()`
  - b. Defines a request schema IrisFeatures using Pydantic with four float inputs
  - c. Creates a GET / endpoint for health check
  - d. Creates a POST /predict endpoint to take input features and return prediction
7. Create a requirements.txt file including: fastapi,uvicorn,joblib,scikit-learn,numpy
8. Write a Dockerfile with the following steps:
  - Use python:3.11-slim as the base image
  - Set working directory to /app
  - Copy and install dependencies from requirements.txt
  - Copy application files (app/ and main.py)
  - Expose port 8000
  - Define command to run FastAPI using uvicorn
9. Build the Docker image: `docker build -t ml-model-api .`
10. Run the Docker container: `docker run -p 8000:8000 ml-model-api`
11. Test the API endpoint /predict using: cURL or Python requests library

## Flow Chart:



## Code:

### train.py — Model Training Script

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
import joblib
import os

def train_and_save_model():
    os.makedirs('app', exist_ok=True)
    iris = load_iris()
    X, y = iris.data, iris.target
    model = RandomForestClassifier()
    model.fit(X, y)
    joblib.dump(model, 'app/model.pkl')
    print("Model trained and saved to app/model.pkl")

if __name__ == "__main__":
    train_and_save_model()
```

### **main.py — FastAPI REST Application**

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
import numpy as np
model = joblib.load("app/model.pkl")
class IrisFeatures(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float
app = FastAPI(title="ML Model API")
@app.get("/")
def home():
    return {"message": "ML Model API is running"}
@app.post("/predict")
def predict(features: IrisFeatures):
    input_data = np.array([[
        features.sepal_length,
        features.sepal_width,
        features.petal_length,
        features.petal_width
    ]])
    prediction = model.predict(input_data)
    return {"prediction": int(prediction[0])}
```

### **Requirements.txt**

```
fastapi
uvicorn
joblib
scikit-learn
Numpy
```

### **Dockerfile**

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ ./app/
COPY main.py .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

### **Build, Run and Test**

```
# Train and save the model
python train.py
# Build the Docker image
docker build -t ml-model-api .
# Run the container
docker run -p 8000:8000 ml-model-api
# Test using cURL
curl -X POST http://localhost:8000/predict \
-H "Content-Type: application/json" \
-d '{"sepal_length":5.1,"sepal_width":3.5,"petal_length":1.4,"petal_width":0.2}'
```

## Output:

```
[+] Building 0.0s (0/0)  docker:desktop-linux
[+] Building 104.3s (11/11) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                  0.1s
=> => transferring dockerfile: 566B                                  0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim  5.2s
=> [internal] load .dockerignore                                     0.0s
=> => transferring context: 2B                                         0.0s
=> [1/6] FROM docker.io/library/python:3.11-slim@sha256:b1b81d67b8df73bf6067191790c233ef9e598b863fc1ccf44  0.1s
=> => resolve docker.io/library/python:3.11-slim@sha256:b1b81d67b8df73bf6067191790c233ef9e598b863fc1ccf44  0.1s
=> [internal] load build context                                     0.1s
=> => transferring context: 190.64kB                                   0.0s
=> CACHED [2/6] WORKDIR /app                                         0.0s
=> [3/6] COPY requirements.txt .                                     0.1s
=> [4/6] RUN pip install --no-cache-dir -r requirements.txt         78.9s
=> [5/6] COPY app/ ./app/                                           0.2s
=> [6/6] COPY main.py .                                             0.1s
=> exporting to image                                               19.5s
=> => exporting layers                                               14.3s
=> => exporting manifest sha256:ed6a697b1c47224045746f7c0a5f8c90324f6aad051abf791923bd27bd4f3032  0.0s
=> => exporting config sha256:a4476bf822ccc8bedfb9b772ffdc097e9fd4f66bd2030e5070bfafbefd420c7c  0.0s
=> => exporting attestation manifest sha256:b4c7802461c208697f999ab4afcc52ec0387828f870aa0265bb6ff3efbc1a  0.0s
=> => exporting manifest list sha256:ed7cc56e33137bb9e718f2ac3bbd3dcafd40c6d3c67dbcee6faa9a849cf1c816  0.0s
=> => naming to docker.io/library/ml-model-api:latest               0.0s
=> => unpacking to docker.io/library/ml-model-api:latest            5.0s

/usr/local/lib/python3.11/site-packages/sklearn/base.py:463: InconsistentVersionWarning: Trying to unpickle estimator DecisionTreeClassifier from version 1.5.1 when using version 1.8.0. This might lead to breaking code or invalid results. Use at your own risk.
For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
/usr/local/lib/python3.11/site-packages/sklearn/base.py:463: InconsistentVersionWarning: Trying to unpickle estimator RandomForestClassifier from version 1.5.1 when using version 1.8.0. This might lead to breaking code or invalid results. Use at your own risk.
For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 172.17.0.1:43980 - "POST /predict HTTP/1.1" 422 Unprocessable Entity
INFO: 172.17.0.1:58924 - "POST /predict HTTP/1.1" 422 Unprocessable Entity
INFO: 172.17.0.1:41974 - "POST /predict HTTP/1.1" 422 Unprocessable Entity
INFO: 172.17.0.1:49958 - "POST /predict HTTP/1.1" 200 OK
```

## Discussion on Results and Conclusion:

This experiment successfully demonstrated the complete workflow of training a machine learning model, deploying it as a REST API, and containerizing the application using Docker. The use of FastAPI provided a lightweight and high-performance API framework. It also enabled automatic request validation using Pydantic models, making the API more reliable and efficient. Separating the training script (train.py) from the serving application (main.py) follows good MLOps practices, as it keeps model development and deployment

independent from each other. A key best practice used in this experiment was copying requirements.txt before the application code in the Dockerfile. This allows Docker to cache dependencies and avoid reinstalling them during every build, which helps in reducing build time. Using python:3.11-slim as the base image also reduced the container size while maintaining full functionality.

**Conclusion:** A Random Forest model trained on the Iris dataset was successfully deployed as a FastAPI-based REST API and packaged into a Docker container. The container was built, executed, and tested successfully, demonstrating a complete and portable machine learning deployment workflow.

### **Learning Outcomes:**

- Understood the importance of containerization in making ML applications portable and environment-independent.
- Learned to structure an ML project directory for clean separation of training, serving, and configuration files.
- Gained hands-on experience in writing a Dockerfile following best practices such as layer caching and slim base images.
- Learned to build and expose ML models as REST APIs using FastAPI and Pydantic for input validation.
- Understood how to serialize and deserialize trained ML models using joblib for deployment.
- Applied core MLOps practices of reproducible environment setup and containerized model serving using Docker.